

产品	YPOX	版本	V1.1
密级	D	页数	14

文件名称：

YPOX-串口通信协议说明书

文件编号：YPOX-2-001 (V1.1)

内容摘要及关键字：

摘要： 本文档描述 YPOX 系列的血压计通信协议的物理定义和软件定义。
关键字： 血压计、通讯协议.

起草：		日期：	
审核：		日期：	
批准：		日期：	

深圳市亿米生命科技有限公司

目录

相关文件.....	3
缩略语.....	3
修改记录.....	3
1 概述.....	4
1.1 编写目的	4
1.2 背景	4
1.3 定义	4
1.4 参考资料	4
2 通信协议.....	4
2.1 功能安全要求	4
2.2 物理协议	5
2.3 软件协议	5
2.4 应答机制	5
3 血压计帧.....	6
3.1 下行帧	6
3.1.1 帧描述.....	6
3.1.2 帧详细说明.....	6
3.2 上行帧	6
3.2.1 帧描述.....	6
3.2.2 帧详细说明.....	7
4 协议说明.....	8
5 操作流程.....	9
6 参考代码.....	10

相关文件

无

缩略语

英文缩写	英文全称	中文
NIBP	Non-invasive Blood Pressure	无创血压

修改记录

文件版本	修改日期	修改人	修改描述历史
V1.0	2024.9.10	刘桂川	首次发布
V1.1	2025.3.5	刘桂川	优化指令

YP0X-串口通信协议说明书

1 概述

1.1 编写目的

本文档详细叙述上位机(或 PC 机)和 YP0X 血压计之间的通信实现。文档定义了血压计与上位机之间的物理连接规范、协议结构及帧操作的意义和帧的有效性。本文档是 YP0X 血压计软件设计需求说明书的一部分。

读者：软件编程人员，软件单元及测试人员，PC 设计人员，上位机设计人员。

1.2 背景

为满足市场需求，独立开发 YP0X 血压计。

1.3 定义

上位机：主要是指监护仪或 APP 应用软件。

下位机：指 YP0X 血压计（以下简称血压计）。

用户帧：用户使用的帧，对厂家和用户开放。

1.4 参考资料

无。

2 通信协议

2.1 功能安全要求

校验：帧须有校验和。

应答：上位机或下位机在接收到应答需求，需返回应答。

容错：要有通讯的容错机制。

2.2 物理协议

电平：TTL 电平。

波特率：115200pbs。

帧格式：1 位起始位 + 8 位数据位 + 1 位停止位 +无校验。

2.3 软件协议

帧协议格式：帧头+ 帧长 + 帧数据（命令/应答+命令数据）+帧校验和。

表格 1 软件协议表

帧头	帧长	帧数据		帧校验和
		命令	命令数据	
Frame header	Frame Length	Command ID	Command Data	Frame CheckSum
Byte1	Byte2	Byte3	Byte4~Byte(N-1)	ByteN
1byte	1byte	1byte	(N-4) byte	1byte

帧域说明：

Frame-Header: 0xFE ， 占 1 个字节。

Frame-Length: Frame-Header+Frame-Length+ Command-ID + Command-Data + Frame-Checksum 的字节长度， 占 1 个字节。

Command-ID: 占 1 个字节。

Command-ID:	Bit0~Bit5:CMD-ID， 请参见具体的帧。 Bit6:0 无应答； 1 需应答
--------------------	---

Command-Data: 请参见具体的帧， 占 N-5 个字节。

Frame-Checksum: Command-ID+Command-Data 的字节累加校验和， 若等 0xFF， 占 1 个字节。

其它说明：

1) N >= 4 && N <=35。

2.4 应答机制

1) 血压计或上位机正确接受应答需求帧后，须在 200ms 内返回应答帧，若发送方在 200ms 内未接收到应答帧，则该帧再发送一次，若连续 3 次发送未应答，则认为帧通信失败。

3 血压计帧

3.1 下行帧

3.1.1 帧描述

上位机对血压计的控制主要通过该部分来实现。下位机接收到有效帧后返回相应的应答包，并进行相应的处理操作。

Command-ID: 0x10~0x14 预留

3.1.2 帧详细说明

3.1.2.1 测量操作(0x10)

帧头	帧长	命令	命令数据	帧校验和	备注
0xFF	3	0x10	Data	Checksum	无应答

1) 数据内容描述

内容	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Data1	0-表示停止测量； 血压计进入休眠状态 1-表示启动测量。							
说明： 只有在血压计非待机下有效								

3.1.2.2 查询(0x13)

帧头	帧长	命令	命令数据	帧校验和	备注
0xFF	3	0x13	Data	Checksum	无应答

2) 数据内容描述

内容	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Data1	0-表示查询版本； 1-表示查询最后一次的血压测量值。							
说明： 血压计在非待机状态下， 查询一次响应一次。								

3.1.3 帧详细说明

3.2 上行帧

3.2.1 帧描述

血压计向上位机发送的帧：包含血压计的状态、测量结果等。

Command-ID: 0x31~0x3F

3.2.2 帧详细说明

3.2.2.1 测量结果(0x31)

帧头	帧长	命令	命令数据	帧校验和	备注
0xFF	10	0x31	Data	Checksum	无应答

3) 数据内容描述

内容	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Data1	测量信息:							
Data2	收缩压值低 8 位							
Data3	收缩压值高 8 位							
Data4	舒张压值							
Data5	平均压值							
Data6	脉率值低 8 位							
Data7	脉率值高 8 位							
Data8	预留;							

说明:

(1) 血压测量一次完成后发送一次。

(2) 收缩压范围【40~255】，单位 mmHg，无效值 0。

(3) 舒张压范围【30~225】，单位 mmHg，无效值 0。

(4) 脉率范围【30~200】，单位 BPM，无效值 0。

(5) 血氧值范围【0~100】，无效值 7F。

3.2.2.2 血压计信息帧(0x39)

帧头	帧长	命令	命令数据	帧校验和	备注
0xFE	06	0x39	Data	Checksum	无应答

1) 数据内容描述

内容	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Data1	Bit0~Bit6: 软件主版本 Bit7: 0							
Data2	Bit0~Bit6: 软件次版本 Bit7: 0							
Data3	Bit0~Bit6: 软件低版本 Bit7: 0							
Data4	Bit0~Bit6: 血压计信息(保留) 0x01: YP01 0x02: YP02 0x03: ...							
说明								
(1) 举例 V01.45.39, 则有: Data2=01, Data3 = 45, Data4= 39								
(2) 在接收到版本查询时, 发送一次								

4 协议说明

- ◆ 血压计按接收的先后次序依次执行帧。
- ◆ 在帧发出后 200ms 如果收不到应答，可以认为发生了通讯错误。
- ◆ 为避免数据用浮点数表示，实际数据值分别乘以 10 或 100 或 1000，使用时应注意各个参数值实际有效范围。
- ◆ 血压计在接收到有效的帧后，返回应答包

5 操作流程

6 参考代码

UART 口配置如下：

```
USART_ImtStructure.USART_BaudRate = 115200;
USART_InitStructure.USART_WordLength=USART_WordLength_8b;
USART_ImtStmcture.USART_StopBits=USART_StopBits_1;
USART_InitStructure.USART_Parity = USART_Parity_No;
USART_InitStructure.USART_HardwareFlowContro=USART_HardwareFlowControl_None
```

// 软件协议宏定义

```
#define FRAME_HEADER          0xFF          // 帧头
#define FRAME_DATA_LEN_MAX    30
#define FRAME_LEN_POS         1             // 帧长位置
#define FRAME_LEN_MIN         2             // 帧长的最小长度值
#define FRAME_LEN_MAX         (FRAME_DATA_LEN_MAX+FRAME_LEN_MIN)           // 接收帧的最大数据个数
#define FRAME_CMD_POS         2             // 帧命令位置
#define FRAME_CMD_DATA_POS    3             // 帧命数据位置
#define FRAME_ACK_POS         2             // 模块 ID 位置
#define FRAME_ACK_MARK        0x40          // 模块 ID 掩码
#define FRAME_ACK_TIME        (1000*SYSTEM_TIME_FREQUENCY/1000)
#define FRAME_ACK_DATA_SIZE   10
#define USART_DATA_BUFF_MAX   30
```

```
typedef unsigned char UCHAR;
```

```
typedef unsigned char BOOL;
```

```
extern s16 HOST_CheckSum(u8 Flg,u8* pData)
```

```
{
    s16 iResult = SM_ERROR;
    u8 ucLen;
    u8 ucCheckSum = 0;
    u8 i;

    if (pData)
    {
        ucLen = pData[FRAME_LEN_POS]+1;
        for (i = FRAME_LEN_POS; i < ucLen; i++)
        {
            ucCheckSum += pData[i] ;
        }
    }
}
```

```

        if (FRAME_HEADER == ucChecksum)
        {
            ucChecksum = ucChecksum - 1;
        }

        if (FALSE == Flg)
        {
            pData[ucLen] = ucChecksum;
        }
        else
        {
            if (ucChecksum == pData[ucLen])
            {
                iResult = SM_NORMAL;
            }
        }
    }

    return iResult;
}

```

```

extern s16 HOST_GetChecksum(u8* pData)
{
    s16 iResult = SM_ERROR;
    u8 ucLen;
    u8 ucChecksum;
    u8 i;

    if (pData)
    {
        ucChecksum = 0;
        ucLen = pData[FRAME_LEN_POS]+1;

        //
        for (i = FRAME_LEN_POS; i < ucLen; i++)
        {
            ucChecksum += pData[i] ;
        }

        if (FRAME_HEADER == ucChecksum)
        {
            ucChecksum = ucChecksum - 1;

```

```

    }

    //
    pData[ucLen] = ucChecksum;

    iResult = SM_NORMAL;
}

return iResult;
}

extern void HOST_UnpackingFrame(void)
{
    static u8 s_ucHostFrameHeadFlg = FALSE;
    static u8 s_ucHostFrameLenFlg = FALSE;
    static u8 s_ucHostCurPos = 0;
    static u8 s_ucHostResetByte = 0;
    static u8 s_ucHostReceData[USART_DATA_BUFF_MAX] = {0};
    static u8 s_ucHostDataBuff[USART_DATA_BUFF_MAX] = {0};
    u8 *pData = s_ucHostReceData;
    u8 ucReadLen;
    u8 ucCurByte;

    // [1.] 猫联路氓联聳盲赂虏氓联拢忙聳撈忙联庐
    ucReadLen = PCUsart_ReadData(pData, USART_DATA_BUFF_MAX);

    // [2.] 猫撈拢忙聳聯忙聳撈忙联庐
    while(0 < ucReadLen)
    {
        ucCurByte = *pData;

        if(TRUE == s_ucHostFrameHeadFlg)    // 氓頻聳氓聳婪氓赂撈氓陇麓
        {
            if(s_ucHostFrameLenFlg)          // 忙拢聳忙碌聳氓聳撈氓赂撈茅聳聳
            {
                s_ucHostDataBuff[s_ucHostCurPos] = ucCurByte;
                s_ucHostCurPos++;
                s_ucHostResetByte--;

                // re
                if(0 >= s_ucHostResetByte)
                {
                    // check sum

```

```

        if (HOST_CheckSum(TRUE, s_ucHostDataBuff))
        {
            // command pro

        }

        // reset
        s_ucHostFrameHeadFlg = FALSE;
        s_ucHostFrameLenFlg = FALSE;
        memset(&s_ucHostDataBuff, 0x0, USART_DATA_BUFF_MAX);
    }
}
else
{
    // 氓赂楼茅聾驴氓脔陇忙聿颇
    if ((FRAME_LEN_MAX >= ucCurByte) && (FRAME_LEN_MIN <= ucCurByte))
    {
        // 忙颇拢氓赂赂盲驴隆忙聿炉
        s_ucHostDataBuff[s_ucHostCurPos] = ucCurByte;
        s_ucHostResetByte = ucCurByte;
        s_ucHostFrameLenFlg = TRUE;
        s_ucHostCurPos ++;
    }
    else
    {
        // 脉脔脉篓脉聿脉聿
        s_ucHostFrameHeadFlg = FALSE;
        s_ucHostFrameLenFlg = FALSE;
    }
}
}

// 氓聿聿氓脔麓忙拢聿忙碌聿
if (FALSE == s_ucHostFrameHeadFlg)
{
    if (FRAME_HEADER == ucCurByte)
    {
        s_ucHostFrameHeadFlg = TRUE;
        s_ucHostFrameLenFlg = FALSE;
        s_ucHostDataBuff[0] = ucCurByte;
        s_ucHostCurPos = 1;
    }
}
}

```

```
    pData++;  
    ucReadLen--;  
}  
}
```